

How to learn long-horizon tasks from few and length-limited training samples

Introduction

Equipping large language models (LLMs) with the ability to execute long-horizon tasks through extended planning and reasoning remains a primary frontier in artificial intelligence. Success in these domains relies on a model's capacity to produce lengthy and diverse reasoning traces. In Appendix A, we show that these kinds of extended reasoning traces are required to accurately solve non-smooth tasks, where minor input changes change the output.

In practice, models already exhibit this behavior; for instance, DeepSeek-R1's reasoning traces for basic grade-school math average $[X]$ tokens. This length introduces a severe sample complexity problem. The space of possible reasoning paths grows exponentially with reasoning length, dwarfing the scale of any feasible pre-training corpus.

It is therefore impossible for training data to meaningfully sample the distribution of valid reasoning sequences. Yet, we would like our LLMs to generalize to novel, long-horizon tasks. In order for this to be possible, different reasoning trajectories must share subsequences in a structured way that increases the sample-efficiency of learning. We propose that this generalization is enabled by local structure within the reasoning traces: the distribution of valid next steps is constrained primarily by recent context, decoupling an individual reasoning step from the full past trajectory. While the LLM has access to the full context, the true transition dynamics of the logical task can be approximated by considering the recent n -gram state, which the model can exploit.

Section 1: Local Structure in Reasoning Traces Enables Generalization with Limited Samples

The complete set of valid reasoning traces for a given task can be organized in a prefix tree. At any node in the tree, outgoing branches represent the set of plausible next reasoning steps. At any given time, the LLM uses the prompt to select the correct next step from this set. Each complete path from root to leaf represents a valid reasoning trace. In Appendix A, we show that by navigating this deep prefix tree, a LLM transforms a single non-smooth task into a sequence of smooth, easy-to-predict intermediate problems, thereby reducing the prediction error through reasoning. Executing this decomposition requires learning every transition in the reasoning prefix tree. We will investigate how a LLM learns all these transitions despite observing only a sparse or length-limited sample of complete paths during training.

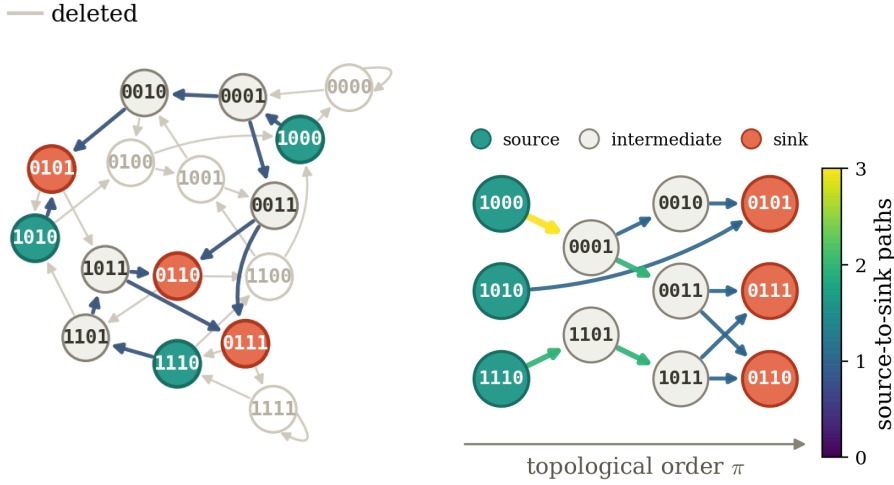


Fig: (Left) De Bruijn graph. Edges show transitions from one 4-gram to another one by adding a single token. Deleted edges are those that were pruned when applying the procedure F to the graph. (Right) DAG created by applying procedure F to the graph on the left with a random node ordering π .

This kind of generalization is possible if reasoning sequences exhibit local structure so that they can be represented by paths on a De Bruijn graph (see fig [X]). A De Bruijn graph $B(V, n)$ consists of a vertex set where each unique n -gram (a sequence of n tokens) from a vocabulary of size V corresponds to a distinct node. Directed edges encode the transitions from one n -gram to the next upon appending a new token. For instance, transitioning over bigrams might look like: "Jane has" $\rightarrow$ "has two" $\rightarrow$ "two apples". If the model only requires the previous n tokens in its reasoning to uniquely determine the set of next plausible tokens, the prohibitively large prefix tree of reasoning traces collapses into a set of unique paths on $B(V, n)$.

Crucially, if we prune all the edges in the De Bruijn graph $B(V, n)$ that do not belong to a valid reasoning path, the resulting subgraph must be a Directed Acyclic Graph (DAG). Including only forward-pointing edges in this way ensures that all reasoning paths are finite.

Remark 1 For the purposes of enumerating reasoning pathways in this paper, we ignore "recovery branches" that point backwards in the graph and allow a LLM to correct incorrect reasoning. Understanding when and where to place such branches for robust error-correction is a compelling avenue for future work.

We would like to understand how much data it would require for a LLM to learn all or most of the transitions in a large DAG.

The De Bruijn graph $B(V, n)$ contains V^n nodes. We designate N vertices as answer nodes with no outgoing edges (corresponding to possible answer choices) and S vertices as start nodes with no incoming edges (the starting sequences). Given a DAG $D_\pi \subset B(V, n)$ that is constructed to maximize the number of distinct start-to-answer paths, we would like to compute (i) the minimum number of samples of reasoning paths required to cover every edge, and (ii) the minimum length of reasoning samples required to cover every edge. The first calculation

demonstrates how few training samples a LLM requires to learn the entire reasoning prefix tree while the second calculation demonstrates how well a LLM can generalize to long-horizon tasks while being trained on mostly short reasoning sequences.

Consider the following procedure F to produce a DAG from a De Bruijn graph $B(V, n)$.

1. Order the nodes according to a permutation π .
2. Designate the last N nodes as answer nodes, and the first S nodes as start nodes. Delete outgoing edges from answer nodes and incoming edges from start nodes.
3. Delete any edge pointing from node u to v if v is not later than u in the ordering. An edge from node u to v remains if and only if $\pi(u) < \pi(v)$.
4. Delete any node and edge that does not lie on a start-to-answer path.

We will represent the action of this procedure on a De Bruijn graph as the function $F[\cdot; \pi, N, S]$ where the first argument is the De Bruijn graph, π is a node ordering on that graph, N is the number of answer nodes and S is the number of start nodes. For example, when this procedure is applied to the graph $B(V, n)$ we write $F[B(V, n); \pi, N, S]$. The sum of the start and answer nodes is constrained by the total number of nodes in the graph. $S + N \leq V^n$.

In the following theorems and their proofs, $B(V, n)$ is a De Bruijn graph of size $V > 4$, $n \geq 2$ and the graph $D_\pi := F[B(V, n); \pi, N, S]$.

Lemma 1 D_π is a DAG for any ordering of nodes π .

Proof:

We will prove, by contradiction, that the generated graph has no cycles. Assume that the graph $F[B(V, n); \pi, N, S]$ contains a cycle. Then, it contains a sequence of nodes $(u_1, u_2, \dots, u_k)$ for some integer $k > 1$ such that there is a directed edge from node u_i to u_{i+1} for any $1 \leq i < k$, and there is a directed edge from u_k to u_1 . This means that $\pi(u_1) < \dots < \pi(u_k) < \pi(u_1) \implies \pi(u_1) < \pi(u_1)$ which is clearly an impossibility. By contradiction, the procedure generates a DAG. $\square$

Lemma 2 There exists a number of start nodes $S > 0$ and a permutation π such that D_π contains the maximal count of distinct paths from a start node to the N answer nodes of any DAG that can be constructed from the edges in $B(V, n)$.

Proof:

Let D^* be a DAG that only contains edges from the De Bruijn graph $B(V, n)$, maximizes the number of distinct paths from its S^* start nodes to the N answer nodes and does not contain any vacuous edges that are not on a start-to-answer path. This DAG has a set ordering of nodes π^* . The first S^* nodes in this ordering will be start nodes with no incoming edges, and the last N nodes in this ordering will be answer nodes with no outgoing edges. Let $D = F[B(V, n); \pi^*, N, S^*]$ be the DAG generated by our procedure using the optimal number of answers S^* and ordering of nodes π^* . This procedure deletes any edge from node u to v such

that $\pi^*(u) \geq \pi^*(v)$. Since π^* is a topological sort of the nodes in D^* , no edge in D^* will be deleted by this procedure. Furthermore, both D and D^* have the same set of start and answer nodes, and D^* does not contain any edges that are not in the de Bruijn graph $B(V, n)$. This means that the generated graph D contains all the edges in D^* , so $D^* \subseteq D$. Therefore the number of distinct paths from start to answer nodes in D is greater than or equal to the number of paths of that kind in D^* . Since D^* is a maximizer of the number of paths, D is as well. $\square$

Lemmas 1 and 2 establish that our procedure F does indeed generate the kinds of reasoning structures we are interested in, i.e., those with a large number of finite-length reasoning traces.

Theorem 1 Let the number of start and answer nodes $S, N < \binom{n+m}{m}$ for an integer $0 < m < (V - 1)/2$. There exists a node ordering π such that

1. the total number of start-to-answer paths in D_π is

$$P_{\text{tot}} \geq \sum_{t=0}^{V-2m} \left[\binom{V-2m}{t} \right]^n$$

2. The minimum number of start-to-answer paths required to cover every edge in D_π is

$$P_{\text{min}} \leq \frac{V^n(V+1)}{2}.$$

Proof:

Define the alphabet of size V , $A = \{0, \dots, V-1\}$. Each node u in D_π can be written as $u = a_1 a_2 \dots a_n$ for all $a_i \in A$. We will also use shorthand such as 0^k or $(V-1)^k$ to refer to length k strings of the repeated character 0 or $(V-1)$ respectively. While this shorthand could be mistaken as exponents, its meaning will be stated or clear from the context. Define the digit-sum of this node u as

$$\sigma(u) = \sum_{i=1}^n a_i.$$

And let π be an ordering by this digit sum such that for any two nodes u_1, u_2 in D_π ,

$$\pi(u_1) < \pi(u_2) \implies \sigma(u_1) \leq \sigma(u_2).$$

(1) We will first show the bound on the total number of start-to-answer paths P_{tot} .

Consider the original De Bruijn graph $B(V, n)$ from which the DAG D_π was derived. There is an edge $u \rightarrow v$ in $B(V, n)$ if and only if we can write $u = a_1 a_2 \dots a_n$ and $v = a_2 a_3 \dots b$ for all $a_i, b \in A$. This edge remains in D_π if $\sigma(v) > \sigma(u)$ so $b > a_1$. Taking this one step further, consider the directed path of length l , $u_1 \rightarrow u_2 \rightarrow \dots \rightarrow u_l$ in $B(V, n)$. This path is generated from the string $a_1 a_2 \dots a_{n+l}$ where each $u_j = a_j a_{j+1} \dots a_{n+j}$. The path is guaranteed to exist in D_π if (i) u_1 is a start and u_n is a answer, (ii) no intermediate node u_i for $1 < i < l$ is a start or a answer, (iii) and every edge in the path points to a node with a greater digit-sum than

the last. The third condition is equivalently rewritten as

$$a_{n+j} > a_j$$

for any $1 \leq j \leq l$. This observation will be useful later in the proof.

Recall the start nodes are the first S nodes in π and the answer nodes are the last N nodes in π . Because the number of start and answer nodes are bounded by $\binom{n+m}{m}$, each start node has digit-sum $\leq m$ and each answer node has digit-sum $\geq n(V-1) - m$. This is because there are exactly $\binom{n+m}{m}$ possible ways to create start or answer nodes with their respective allowed digit-sums (by the stars and bars method). In addition, the node with the smallest digit-sum 0^n is guaranteed to be a start and the node $(V-1)^n$ with the largest digit-sum is guaranteed to be a answer node. Therefore, the number of distinct paths from 0^n to $(V-1)^n$ in D_π is a lower bound on the total number of start-to-answer paths in D_π . We would like to estimate this value.

A generic path between these two nodes $0^n \rightarrow u_1 \rightarrow \dots \rightarrow u_{tn} \rightarrow (V-1)^n$ with tn intermediate nodes can be generated from the string $0^n a_1 a_2 \dots a_{tn} (V-1)^n$. This is a valid path in D_π if

- (i) $a_i \notin \{0, 1, \dots, m\} \cup \{V-1, V-2, \dots, (V-1) - m\}$ which guarantees that no intermediate node in the path is a start or a answer,
- (ii) $a_{j+n} > a_j$ for any $1 \leq j \leq n(t-1)$.

These conditions are repeated from our observation above. Satisfying them requires that $0 \leq t \leq (V-2m)$. The number of such paths is just the number of monotonically increasing sequences of length t , raised to the n and summed over all t

$$\sum_{t=0}^{V-2m} \left[\binom{V-2m}{t} \right]^n.$$

(2) Now we will prove the second statement in the theorem for the same ordering π . First note that the minimum number of paths required to cover every edge in D_π , denoted by $P_{\min}$, is at most the number of edges in D_π . The following argument proves this fact.

Consider the set $\mathcal{P}_{\min}$ of paths in D_π that cover all the edges in D_π and is of size $P_{\min}$. Each path $p \in \mathcal{P}_{\min}$ must contain at least one edge in D_π that is not in any other path in $\mathcal{P}_{\min}$. Otherwise the set $\mathcal{P}_{\min} \setminus \{p\}$ has size $P_{\min} - 1$ and still covers every edge in D_π . This is in contradiction with the definition that $P_{\min}$ is the minimum number of paths required to cover all the edges in D_π . Since every path $p \in \mathcal{P}_{\min}$ is associated with at least one unique edge, $P_{\min}$ can be at most the number of edges in D_π . We would like to upper bound the number of edges in D_π which will give us an upper bound on $P_{\min}$.

For every pair of nodes $u = a_1 \dots a_n$ and $v = b_1 \dots b_n$ for $a_i, b_i \in A$, an edge exists from $u \rightarrow v$ only if $b_n \geq a_1$ and $a_{i+1} = b_i$ for all $1 \leq i \leq n-1$. There are

$$\frac{V^n(V+1)}{2}$$

ordered pairs of nodes satisfying these properties which is an upper bound on $P_{\min}$. $\square$

Corollary 1 The ratio $P_{\min}/P_{\text{tot}} = 0$ in the limit $V \rightarrow \infty$. In addition, $P_{\min}/P_{\text{tot}} = 0$ in the limit $n \rightarrow \infty$ provided V is sufficiently large that $\binom{V-2m}{\lfloor (V-2m)/2 \rfloor} > V$.

Proof:

By taking the largest term in the bound on P_{tot} in the first part of theorem 1, we know that

$$P_{\text{tot}} \geq \left[\binom{V-2m}{\lfloor (V-2m)/2 \rfloor} \right]^n.$$

In the asymptotic limit $V \rightarrow \infty$ we apply Stirling's approximation and get

$$P_{\text{tot}} \geq \left(\frac{2}{\pi} \right)^{n/2} \frac{2^{n(V-2m)}}{(V-2m)^{n/2}}.$$

In this limit, the ratio $P_{\min}/P_{\text{tot}}$ decays to 0 exponentially quickly as V increases.

Next consider the same ratio

$$\frac{P_{\min}}{P_{\text{tot}}} \leq \left[\frac{V+1}{2} \left(\frac{V}{\binom{V-2m}{\lfloor (V-2m)/2 \rfloor}} \right)^n \right]$$

in the limit that $n \rightarrow \infty$. For sufficiently large V , the fraction $V/\binom{V-2m}{\lfloor (V-2m)/2 \rfloor} < 1$. Thus, taking the limit $n \rightarrow \infty$ of $P_{\min}/P_{\text{tot}}$ yields 0. $\square$

Corollary 1 shows that when reasoning traces are organized on a De Bruijn graph structure, it is possible to learn a large number of reasoning paths despite seeing very little data.

Theorem 2 Let the number of start and answer nodes $S, N < \binom{n+m}{m}$ for an integer $0 < m < (V-1)/2$. There exists a node ordering π such that,

1. The longest start-to-answer path in D_π has length $L_{\max} \geq n(V-2m-1)$.
2. Every edge in D_π lies on a start-to-answer path of length $L_{\text{cover}} \leq 2n+1$.

Proof:

Let π be the digit-sum ordering as described in the proof of Theorem 1. If two nodes

$u = a_1 a_2 \dots a_n$ and $v = b_1 b_2 \dots b_n$ have the same digit-sum, we will break ties by

lexicographically comparing their respective reversed strings $\bar{u} = a_n a_{n-1} \dots a_1$ and

$\bar{v} = b_n b_{n-1} \dots b_1$. Specifically, $\bar{v} < \bar{u}$ in the lexicographic comparison if for

$i^* = \max\{i : a_i \neq b_i\}$, $b_{i^*} < a_{i^*}$. If $\bar{v} < \bar{u}$ lexicographically and both nodes have the same digit-sum, then $\pi(v) > \pi(u)$.

(1) We will start by proving the first bound about the longest start-to-answer path in the

statement of the theorem. We can construct a path $u_1 \rightarrow \dots \rightarrow u_L$ based on the string

$$w = w_1 \dots w_{n(V-2)} = 0^n(m+1)^n(m+2)^n \dots (V-2-m)^n(V-1)^n$$

where each node $u_j = w_j w_{j+1} \dots w_{j+n-1}$ is the associated slice of the string. The digit-sum is strictly increasing along this path, $\sigma(u_{j+1}) - \sigma(u_j) > 0$, the first node $u_1 = 0^n$ is guaranteed to be a start and the last node $u_L = (V-1)^n$ is an answer. In addition, recall from the proof of Theorem 1 that every start node s has digit-sum $\sigma(s) \leq m$ and every answer node s' has digit-sum $\sigma(s') \geq n(V-1) - m$, so no intermediate node u_j for $1 < j < L$ is a start or an answer node. Therefore, the path $u_1 \rightarrow \dots \rightarrow u_L$ is contained in D_π where $L = n(V-2m-1) + 1$. This path has length $n(V-2m-1)$, proving the first bound in the theorem.

(2) Next we will prove the second bound in the theorem about the length of paths required to cover every edge in D_π . Let $u \rightarrow v$ be an arbitrary edge in D_π . We will show that this edge is contained in a path of length less than or equal to $2n + 1$.

We start by proving that there exists a path from a start node to $u = a_1 a_2 \dots a_n$ that has length less than or equal to n . Consider a sequence of $n + 1$ nodes $(u_0, u_1, \dots, u_n)$ such that $u_0 = 0^n$, $u_n = u$ and each intermediate $u_j = 0^{n-j} a_1 a_2 \dots a_j$. Let i_0 be the index of the latest start node in the sequence. We will prove that the path $u_{i_0} \rightarrow u_{i_0+1} \rightarrow \dots \rightarrow u_n$ is a valid path in D_π of length $n - i_0$. If u is a start node, then the path is of length 0 and is trivially in D_π . In addition, no node in the path is an answer node since u has an outgoing edge to v and the intermediate nodes have a digit-sum of at most $(n-1)(V-1)$ which is smaller than the minimum answer node digit-sum of $n(V-1) - 1$. Consider a transition $u_{j-1} \rightarrow u_j$ in the path for $j > i_0$. There are two cases.

(i) If $a_j > 0$, then $\sigma(u_j) > \sigma(u_{j-1})$ so $\pi(u_j) > \pi(u_{j-1})$ and the edge $u_{j-1} \rightarrow u_j$ exists in D_π .

(ii) If $a_j = 0$, we can compare u_j with u_{j-1} using the reverse lexicographic sort. Let $i^* = \max\{i : i < j \text{ and } a_i > 0\}$ which is well-defined for any u_j in the path

$u_{i_0} \rightarrow u_{i_0+1} \rightarrow \dots \rightarrow u_n$ since $u_j \neq 0^n$ for any $j > i_0$. The character a_{i^*} is at position $n - j + i^* + 1$ in node u_{j-1} and it is at the earlier position $n - j + i^*$ in node u_j with every subsequent character being 0, which is smaller than a_{i^*} . This means $\bar{u}_j < \bar{u}_{j-1}$ in the reversed lexicographic sort which we are using for tie-breaking, so $\pi(u_j) > \pi(u_{j-1})$ and, once again, the edge $u_{j-1} \rightarrow u_j$ exists in D_π .

Therefore, the entire path $u_{i_0} \rightarrow u_{i_0+1} \rightarrow \dots \rightarrow u$ is in D_π . This means there exists a path from a start node to an arbitrary node u in D_π that has length less than or equal to n .

Next, we prove that there exists a path from an arbitrary node $v = b_1 b_2 \dots b_n$ in D_π to an answer node that has length less than or equal to n . There are two cases.

(i) Each $b_j < (V-1)$. Consider a sequence of $n + 1$ nodes $(v_0, v_1, \dots, v_n)$ such that

$v_n = (V-1)^n$ is the largest possible node, $v_0 = v$ and each intermediate

$v_j = b_{j+1} b_{j+2} \dots b_n (V-1)^j$. Let k_0 be the index of the earliest answer node in the sequence.

We will prove that the path $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{k_0}$ is a valid path in D_π . If v is an answer node, then the path has length 0 and is trivially in D_π . In addition, no node in the path is a start

node since v has an incoming edge from u and the intermediate nodes have a digit-sum of at least $V - 1$ which is larger than the maximum start node digit-sum of 1. Consider a transition $v_{j-1} \rightarrow v_j$ in the path. Since $b_j < V - 1$, $\sigma(v_j) > \sigma(v_{j-1})$ so the edge $v_{j-1} \rightarrow v_j$ exists in D_π . Therefore, the path $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_{k_0}$ is a valid path in D_π of length less than or equal to n .

(ii) There exists a j such that $b_j = V - 1$. Let $i_1 = \min\{i : b_i = (V - 1)\}$. Consider a path $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_m$ for $v_0 = v$ and to a answer node v_m . This path can be generated by a string $b_1 b_2 \dots b_{n+m}$ where each $v_j = b_{j+1} \dots b_{j+n}$. We will prove by contradiction that $m \leq i_1$, otherwise the path cannot exist in D_π . Consider $m > i_1$ and assume the path exists in D_π . There is a transition $v_{i_1-1} \rightarrow v_{i_1}$ in the path. This transition can be written in characters as $(V - 1)b_{i_1+1} \dots b_{n+i_1-1} \rightarrow b_{i_1+1} \dots b_{n+i_1}$. Since the transition is represented by an edge in D_π , it must be true that

$\sigma(v_{i_1-1}) \leq \sigma(v_{i_1}) \implies (V - 1) \leq b_{n+i_1} \implies b_{n+i_1} = (V - 1) \implies \sigma(v_{i_1-1}) = \sigma(v_{i_1})$, resulting in a tie for the digit-sum. Comparing the reverse lexicographic sort yields

$\bar{v}_{i_1} > \bar{v}_{i_1-1} \implies \pi(v_{i_1}) < \pi(v_{i_1-1})$ provided $v_{i_1} \neq (V - 1)^n$ is not a answer (true by assumption). This means there cannot be an edge from $v_{i_1-1} \rightarrow v_{i_1}$ contradicting our assertion that the path $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_m$ exists in D_π . Therefore it must be true that $m \leq i_1 \leq n$. It follows that, if there exists a path in D_π from v to a answer node (true by assumption that the edge $u \rightarrow v$ exists in D_π), its length is upper-bounded by n .

If the edge $u \rightarrow v$ exists in D_π , then there is a path from a start node to u of length $\leq n$ and there is a path from v to a answer node of length $\leq n$ in D_π . The walk combining these two paths with the edge $u \rightarrow v$ only contains forward-pointing edges in π and is therefore a valid start-to-answer path in D_π . Thus, any edge $u \rightarrow v$ lies on at least one start-to-answer path of length $\leq 2n + 1$.

Corollary 2 In the limit $n \rightarrow \infty$, the ratio $L_{\text{cover}}/L_{\text{max}} \leq 2/(V - 2m - 1)$.

Proof:

A straightforward evaluation of the limit $\lim_{n \rightarrow \infty} \frac{2n+1}{n(V-2m-1)}$.

Remark 2 For a constant state space V^n , maximizing the vocabulary size V minimizes the ratio $P_{\text{min}}/P_{\text{tot}}$, i.e., it maximizes the fraction of the reasoning prefix tree that can be reconstructed per sample. This is because P_{tot} grows at least combinatorially in V and P_{min} grows at most polynomially in V . It also maximizes the ration $L_{\text{cover}}/L_{\text{max}}$. This seems to suggest large vocabularies are optimal for sample efficiency, contradicting the empirical preference for intermediate vocabulary sizes calibrated to model size and data quantity

(<https://arxiv.org/abs/2407.13623>). This discrepancy is resolved by considering the the complexity of each next-token prediction. The out-degree of any node in D_π increases with V , and larger V increases prediction error by forcing the LLM to distinguish between a large number of plausible next options at every reasoning step (see Appendix A for a more in-depth analysis of this kind of error). The optimal vocabulary size is therefore governed by a trade-off between maximizing the sample efficiency of reconstructing the reasoning prefix tree (favoring

a large V) and minimizing single-step reasoning errors (favoring a small V). With more data, the LLM has smaller single-step reasoning errors, and it can reason most effectively with larger vocabulary sizes V .

This result may also help explain why human experts develop large and niche vocabularies in their respective fields - it allows them to generalize to long-horizon tasks despite efficiently only storing the statistics of short sub-tasks in their memory

(<https://pubmed.ncbi.nlm.nih.gov/28787239/>). This was also studied in humans in chess - the differentiating factor between stronger and weaker chess players was their "vocabulary" of known board positions

(https://iiif.library.cmu.edu/file/Simon_box00005_fld00354_bdl0001_doc0001/Simon_box00005_fld00354_bd).

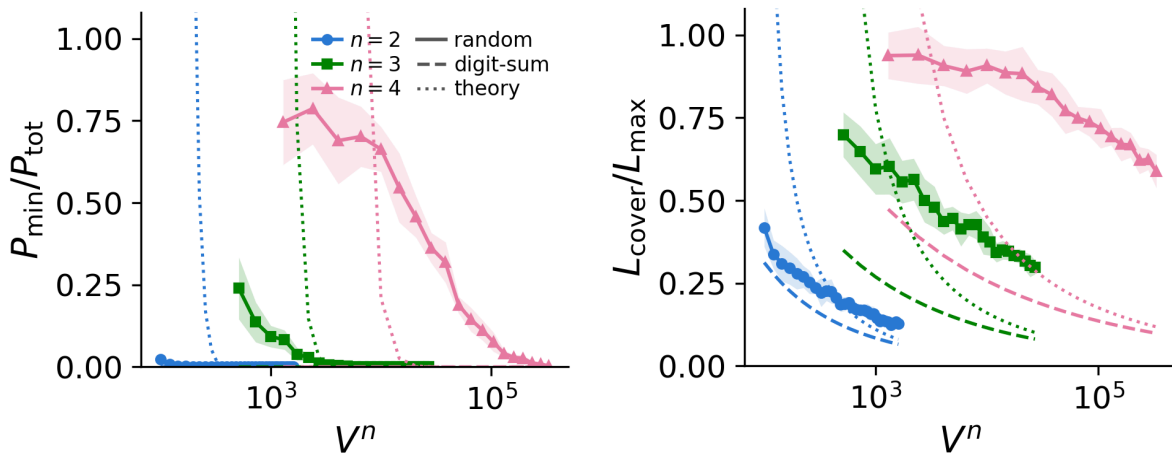


Fig: Plots showing corollary 1 and 2 for particular graphs. Digit-sum is a particular ordering of π , random is the mean and stdev over random orderings, and the dotted line is the theoretical bound for the digit-sum ordering. Random topological sorts increase sample-efficiency, but choosing a particular order is most efficient.

Section 2: There is more uncertainty in the early phases of reasoning compared to later

It is interesting to understand the structure of the reasoning prefix tree that is generated by applying the procedure F to the De Bruijn graph $B(V, n)$.

Theorem 3 In expectation over node orderings π sampled uniformly at random,

$$\mathbb{E}_{\pi}[\deg^+(\pi^{-1}(d))] \leq V \left(1 - \frac{d}{V^n}\right)$$

where $\deg^+(u)$ is the number of outgoing edges for node u and d is the node's position in the sort.

Proof:

We will adopt the notational conventions from the proof of Theorem 1, where A is the alphabet

of size V , and $a \in A$ are characters. Each node u in $B(V, n)$ can be written as $a_1 a_2 \dots a_n$ where each $a_i \in A$. The out-neighbors of u in $B(V, n)$ are $v_b = a_2 \dots a_n b$ for any $b \in A$. There are V such neighbors. Let $c(u)$ count the outgoing branches of a node u in $B(V, n)$. We exclude self-loops from this count since those loops do not survive the DAG D_π cannot contain any. Therefore, $c(u) = V - 1$ for nodes of the form $u = a^n$ since there is one edge from u to itself. There are V such nodes. For all other nodes, $c(u) = V$. Averaged over both kinds of nodes,

$$\frac{1}{V^n} \sum_{u \in B(V, n)} c(u) = \frac{V(V-1) + V(V^n - V)}{V^n} = \frac{V^n - 1}{V^{n-1}}.$$

In addition, for any node $v \neq u$, the probability that $\pi(v) > \pi(u)$ is

$$\Pr[\pi(v) > d | \pi(u) = d] = (V^n - d) / (V^n - 1)$$

by exchangeability of the nodes v . Consider a DAG D'_π which is generated by applying a modified procedure F' that is identical to F without the steps 2 or 4 which delete edges from answer, start and disconnected nodes. In this modified DAG D'_π , the out-degree of the node at position d , $\deg_{\text{modified}}^+(\pi^{-1}(d))$ in expectation over random orderings π is simply the product

$$\mathbb{E}_\pi[\deg_{\text{modified}}^+(\pi^{-1}(d))] = \frac{V^n - 1}{V^{n-1}} \cdot \frac{V^n - d}{V^n - 1} = V \left(1 - \frac{d}{V^n}\right)$$

due to linearity of expectation. Since $D_\pi \subseteq D'_\pi$ (steps 2 and 4 in F delete edges), the quantity above serves as an upper bound on the expected out-degree of the nodes in D_π ,

$\mathbb{E}_\pi[\deg^+(\pi^{-1}(d))]$. $\square$

The initial "exploration" phase of reasoning generally has more uncertainty than later phases. We can think of this initial phase as setting the problem-solving approach for the rest of the reasoning sequence, after which the steps are more straightforward to determine. In actual LLMs, the initial reasoning steps also have much more uncertainty compared to later steps. Once the LLM finds an appropriate strategy to solve the problem, it converges to a solution (see the following figure from my previous paper)

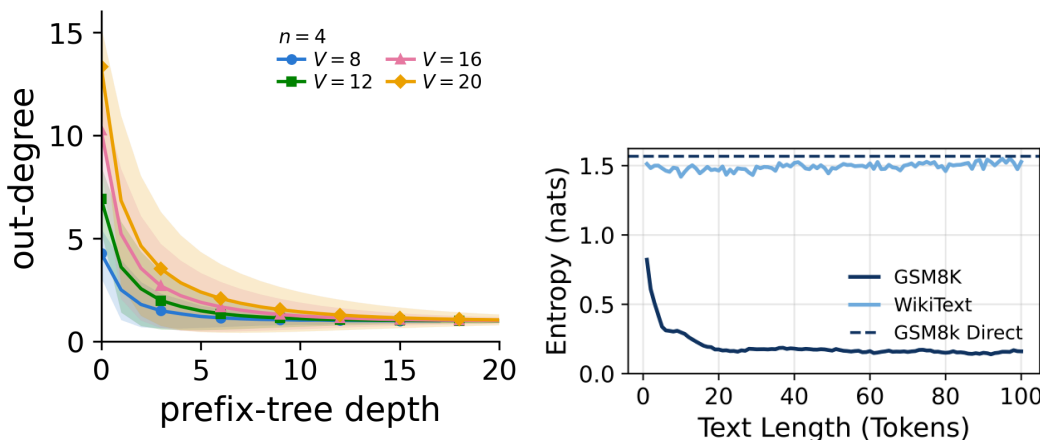


Fig: (Left) Average out-degree of the unrolled prefix tree versus depth over random samples of the topological sort π . answer nodes, with an out-degree of zero, are not counted in the

average. (Right) Perplexity when reasoning decays over time, indicating there is greater uncertainty in the initial reasoning steps.

In discussion:

Make connection to requiring a finite automaton to learn with finite memory.

Difference between effective CoT and honest CoT.

Relation to world-modeling. The recent n -token context represents a finite state-variable. For tasks with this world-modeling structure, it would make sense to include the state variables in the context.

You should study recovery paths by including cycles in the de bruijn graphs. Similar to how it is done in robotics and self-driving car, where if the model makes a wrong or slightly sub-optimal decision, it still has a path to recover from its mistake and continue reasoning. Cycles can induce infinite loops, where a model gets stuck, but also allows for robustness to errors. When/where to include cycles is a direction for future research. It is also interesting to study how using external verifiers can trigger these loops or modify the structure of the tree.

Our theory relies on there being local structure in the reasoning such that it should be traversable with a limited memory. Real-world reasoning tasks likely do not have such a clean De Bruijn structure, but has a more hierarchical one. Many "easier" or more algorithmic problems should be solvable with only 2-gram or 3-gram correlations. While more difficult problems might require learning larger correlations. In addition, models which have learned the larger correlations should be able to reason using fewer steps. This suggests the existence of a much deeper hierarchical structure in real-world reasoning.

While we used de bruijn graphs to represent discrete reasoning transitions, the paths in these kinds of graphs show rich dynamical structure. This indicates that LLMs which learn these paths could have implicitly learned to represent a variety of dynamical time series (see ICML workshop paper). The kinds of time series used to train robots may have the same kind of structure, where most of the next action is predicted using only a limited past context.

reasoning hierarchy (early phase corresponds to more uncertain, higher-level decisions before converging on a strategy)

A key ingredient in recursive self-improvement is a model self-improving the structure of its own reasoning traces to exploit properties like the one explored in this text. This is akin to how humans improve our reasoning when we sleep.

This paper helps introduce what could be a rich and deep field of research related to reasoning. There are many directions in which to take this kind of de bruijn graph analysis of reasoning paths.

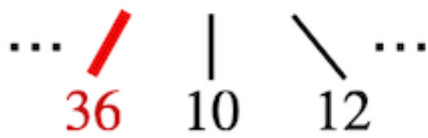
Section 3: Experiments

1. Synthetic experiment. Consider a student-teacher classification problem where the input "context/prompt" is some vectors lying on the surface of a sphere and the output is one of N numbers. Many different combinations of regions on the spheres can map to the same value of N numbers, creating a non-smooth task. These kinds of tasks require large reasoning trees to solve (see Appendix A). Create reasoning traces from an artificial De Bruijn DAG with N answer nodes and maybe $\sim N/2$ start nodes. Each region in the input space can be associated with maybe 5 different reasoning traces sampled from this graph so the model can learn some shared structures between inputs. Show that training with the De Bruijn reasoning traces increases sample-efficiency (plot accuracy vs. num_samples) beyond training with random numerical reasoning traces or a generic prefix tree with no structure or direct classification.
2. In a LLM on math reasoning benchmarks, test how limiting the context to only the last n tokens + the prompt impacts accuracy (attention-knockout experiment). This is despite them being trained with their full context.
3. Take existing LLM reasoning traces from self-play (use majority vote to select more correct ones). Train the model to reproduce its own reasoning while masking the older context. If this doesn't work include periodic prompts for the model to summarize what it knows and then do the same kind of training.
4. Plot perplexity over time for different reasoning benchmarks, not just GSM8k. Show that it decays.
5. See how accuracy is impacted if a big model (of the same family) does the first half of reasoning and a smaller model does the second half. If uncertainty decreases over time, then the first part of the reasoning should be more important to get right.

Appendix A: Predicting non-smooth maps between prompts and answers requires deep reasoning

Consider a task where an LLM is given a prompt and selects one of many possible answers. This is a classification problem, which the model solves by predicting a probability distribution over the next token (this is a simplification, since many answers are multiple tokens long). When LLMs reason using CoT, they are solving a sequence of such classification problems where, at each step, the model must choose the correct next token from a limited set of plausible options.

(i) in a single step

$$(2 + 2) \times (5 + 4)$$


... 36 10 12 ...

(ii) in a sequence (reasoning

$$(2 + 2) \times (5 + 4)$$


4 x 9 36 ...

Fig: The set of plausible reasoning sequences **forms a tree**

Say this task has N equally likely plausible outputs, e.g., AIME problems require an integer between 0 and 999, so $N = 1000$. Then, the input domain can be separated into at least N regions that correspond to each output. As N increases, the number of these regions in the input domain does as well, their volume decreases, and they require correspondingly more data to resolve.

Remark The prompts to language models typically come in the form of discrete sentences, but for the purposes of this analysis we think of prompts as embedded in a continuous input space. This modeling choice is justified by a notion of "similarity" between prompts that indicate the presence of an underlying metric structure, such as that in semantic spaces. This kind of structure is required for generalization to be possible at all. In addition, the total number of possible prompts is significantly larger than the amount of data available for training. This means the set of points in the input domain corresponding to all possible prompts is much more dense than the set of data points, to the point where it may as well be continuous.

Nadgir et al. proved using Lipschitz arguments that the expected probability of error of a classification task is a power law with respect to the number of distinct regions in the input domain for that task, proportional to N

$$E_{\text{direct}} \propto \left(\frac{N^2}{D} \right)^{1/d}$$

where D is the number of i.i.d. data samples and d is the dimension of the input space.

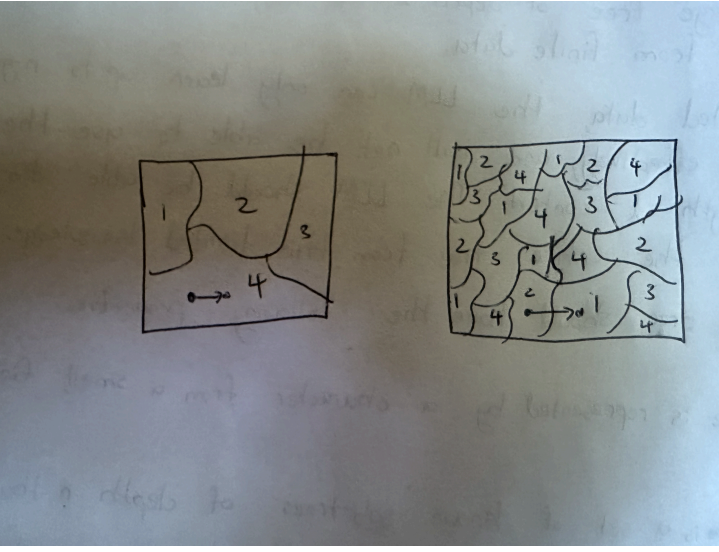


Fig: When there are many regions corresponding to any single answer in the input domain, a small change in the input can cause the output to change.

We consider a closely related setting where the reasoning task is non-smooth such that for every output, there are M distinct regions in the input domain that correspond to that output (see fig). Then, the LLM must distinguish between MN distinct regions in the input domain to determine the correct output class. The corresponding classification error is

$$E_{\text{direct, non-smooth}} \propto \left(\frac{M^2 N^2}{D} \right)^{1/d}.$$

This error can be quite large if M is. CoT helps reduce the error by decomposing this classification problem into a sequence of problems that can each be solved with smaller error. Consider a LLM which generates a CoT of length $\ell - 1$ before predicting the final answer at step ℓ . At any step j in its reasoning sequence it has a choice of, say m_j plausible next tokens. The total number of distinct reasoning paths is denoted by $K = \prod_{j=1}^{\ell} m_j$.

The number of distinct reasoning paths K must be larger than the the number of outputs N but smaller than the number of distinct regions in the input space MN . If the number of distinct paths K were larger than MN , we would be separating the input space into more regions than necessary for prediction. Each reasoning path corresponds to, on average, $(MN)/K$ distinct regions in the input space. We would like to find the optimal value for the number of reasoning paths, K . Applying the scaling law we derived above to each step tells us that the error of a step $j \sim (MNm_j/K)^{2/d}$ since there are m_j plausible outputs, each of which corresponds to $(MN)/K$ regions in the input space.

Applying the union bound to compute the error of reasoning over many steps yields

$$E_{\text{reason, non-smooth}} \leq cD^{-1/d} \left(\frac{MN}{K} \right)^{2/d} \sum_{j=1}^{\ell} m_j^{2/d}$$

for a constant of proportionality c .

We can optimize this bound with respect to the number of reasoning paths K , the number of plausible next tokens at each step m_j and the reasoning depth ℓ , subject to the constraint that the total number of paths $\prod_{j=1}^{\ell} m_j = K$. The optimum occurs when each $m_j = \exp(d/2)$ for any step j . In addition, the number of reasoning paths K should be as large as possible so $K = MN$. The optimal reasoning depth $\sim \log K = \log M + \log N$. Therefore, if the true task is not smooth (M is large), the number of distinct reasoning paths is also large. To learn these long and diverse reasoning paths from limited data, the model must rely on local structure in the reasoning.